

AEGIS

Safety-Gated Autonomous Network Recovery

Creation Roadmap, Team Responsibilities & Technical Curriculum

TEAM: Hrishi • Sahil • Yyash • Vikash

5 September 2026 → 8 September 2026 (qualification-dependent)

CORE PROJECT STATEMENT

Aegis is a simulated autonomous network-recovery engine in which AI diagnoses complex failures and proposes recovery strategies, a digital twin tests those strategies, and deterministic safety policies decide what may be deployed.

The engineering rule: NO AI OUTPUT TOUCHES THE NETWORK DIRECTLY.

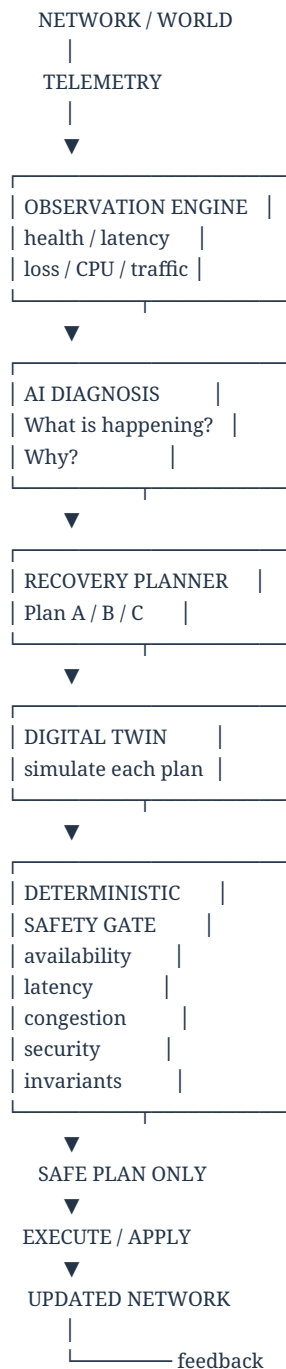
1. What We Are Actually Building

Before anyone writes serious code, the entire team must have the same physical picture of Aegis. We are not attempting to build a production-grade internet network in three days. We are building a technically credible network simulator and autonomous recovery engine that behaves like a small resilient infrastructure system.

SCOPE BOUNDARY

Build a simulated network / digital twin with 15-30 nodes, realistic telemetry, controlled fault injection, AI-generated recovery plans, simulation-based validation, deterministic safety gates, and a real-time dashboard. Real SDN, Kubernetes, cloud, or hardware integrations belong to future scope.

1.1 The end-to-end mental model



THE CENTRAL DESIGN IDEA

AI has intelligence but does not have unrestricted authority. The AI proposes; the digital twin tests; deterministic policies decide whether the proposal is allowed.

1.2 What the MVP must contain

A. Network simulator - A graph of roughly 15-30 nodes. Each node has CPU utilization, memory utilization, health, traffic, latency, packet loss, and status. Each edge has latency, bandwidth, packet loss, and status.

B. Telemetry engine - Continuously produce the current network state. Example: N7 CPU 94%, latency 280 ms, packet loss 17%, traffic 92%.

C. Fault injector - Buttons or commands such as Kill Node, Overload Node, Break Link, Traffic Spike, and Network Partition.

D. AI diagnosis - Given network state + telemetry + topology, return a structured diagnosis, likely cause, affected services, and severity.

E. AI recovery planner - Generate multiple candidate plans using a restricted action language such as reroute, drain_node, migrate_service, quarantine_node.

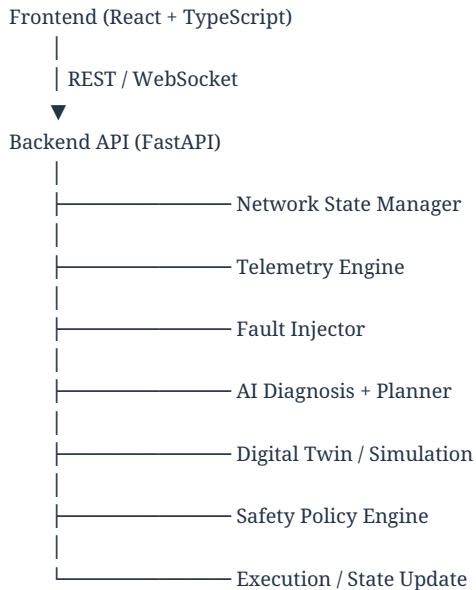
F. Digital twin - Clone the current network state, apply candidate Plan A/B/C to the clone, simulate consequences, and compute outcome metrics.

G. Safety engine - Deterministic rules reject unsafe plans. Example: never disconnect the only client-to-service path; availability must remain above 99%; never route through quarantined nodes.

H. Real-time visualization - Show the topology, node health, telemetry, diagnosis, candidate plans, simulation results, safety decisions, and final recovery.

2. Architecture & Technical Pipeline

The architecture is deliberately modular. Each module has a clear input/output contract so four people can build in parallel without constantly rewriting each other's work.



2.1 State as the single source of truth

Start the project with a single, explicit Network State representation. Every subsystem should consume or transform this state rather than inventing its own representation.

```
{
  "nodes": {
    "N1": {
      "cpu": 32,
      "latency": 12,
      "packet_loss": 0.01,
      "status": "healthy"
    },
    "N2": {
      "cpu": 91,
      "latency": 67,
      "packet_loss": 0.04,
      "status": "degraded"
    }
  }
}
```

WHY THIS MATTERS

If the network simulator, AI, simulator, safety engine, and frontend all agree on one state format, integration becomes dramatically easier. This is one of the first things to stabilize tonight.

2.2 Network representation

Model the network as a weighted graph $G = (V, E)$. Nodes represent servers/routers/services and edges represent communication links. Edge weights can be dynamic rather than fixed.

Static idea:
edge_weight = distance

Aegis idea:
 $W = a * \text{Latency} + b * \text{Congestion} + c * \text{PacketLoss} + d * \text{SecurityRisk}$

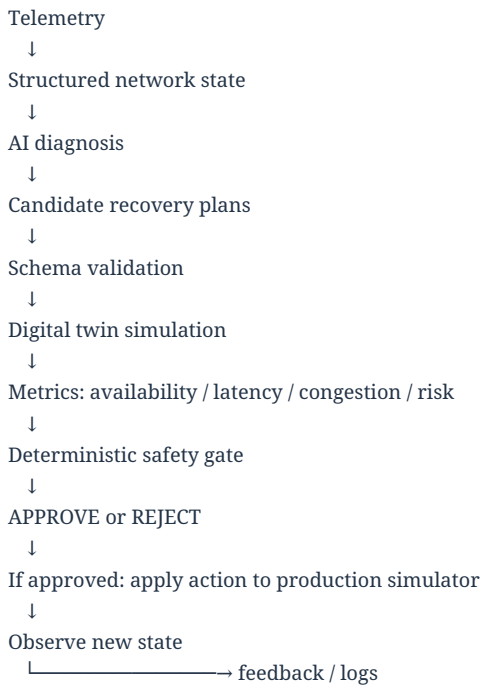
Example:
Route A: latency 20 ms, security risk HIGH
Route B: latency 30 ms, security risk LOW
Aegis may choose Route B because resilience is multi-objective.

2.3 Recovery action language

The AI should never emit arbitrary executable commands. Give it a small, schema-validated action vocabulary. This makes the agent safer and the simulator easier to implement.

```
{
  "actions": [
    {"type": "reroute", "from": "N1", "to": "N9"},
    {"type": "drain_node", "node": "N7"},
    {"type": "migrate_service", "service": "S2", "target": "N11"}
  ]
}
```

2.4 Complete runtime pipeline



3. Technology Stack

Layer	Recommended technology	Why
Language	Python	Fast development; useful ecosystem for graphs, simulation, ML, APIs.
Backend API	FastAPI	Simple Python API layer plus WebSocket support for live updates.
Network graph	NetworkX	Convenient weighted graph model and shortest-path/routing algorithms.
Simulation	SimPy or a custom deterministic simulator	Discrete-event simulation if needed; a custom state-transition simulator is

		acceptable for the MVP.
AI	LLM API	Diagnosis and recovery-plan generation with structured output.
ML	scikit-learn	Lightweight anomaly detection / classification without spending hackathon time training large neural networks.
Validation	Pydantic + custom policy engine	Schema validation plus deterministic safety constraints.
Frontend	React + TypeScript	Fast UI development with a typed state model.
Graph UI	React Flow or Cytoscape.js	Real-time topology visualization.
Realtime	WebSockets	Push node/telemetry/recovery updates to the dashboard.
Containers	Docker	Reproducible local environments and isolated components.
Version control	Git + GitHub	Parallel team development and integration.

DO NOT OVER-ENGINEER

Do NOT introduce Kubernetes, Kafka, Redis, PostgreSQL, cloud deployment, real hardware networking, reinforcement learning from scratch, or a large custom neural network into the first MVP. The core system already has enough technical depth.

4. Team Structure & Ownership

Assign functional ownership, not isolated silos. Each member must understand the whole Aegis pipeline, but one person is ultimately responsible for each subsystem.

Member	Primary ownership	Responsibilities	By 6 Sept
Hrishi	System Architect + Integration Lead	Overall architecture; state model; API contracts; recovery workflow; integration branch; safety philosophy; final reliability; technical narrative.	Define interfaces and end-to-end pipeline; integrate modules.
Sahil	Network Simulation + Digital Twin	NetworkX topology; node/edge state; traffic; latency; failures; state cloning; digital twin; recovery outcome metrics.	Working 15-30 node simulator with fault injection and simulation.
Yyash	AI Diagnosis + Recovery Planner	Anomaly detection; LLM integration; network-state summarization; structured prompts; plan generation; ranking; AI explanations.	Working structured diagnosis + 2-3 valid recovery plans.
Vikash	Safety Engine + Frontend	Deterministic policy rules; constraint validation; plan approval/rejection; risk display; dashboard; topology UI; WebSocket integration.	Live dashboard + functioning safety gate.

TEAM RULE

Nobody should be the only person who understands a critical component. Pair-review interfaces and do a short handoff walkthrough before any merge that changes a shared contract.

5. 5 September - Common Technical Curriculum

Today is the foundation day. The goal is not mastery of every technology; it is a shared technical mental model. Everyone should be able to explain the following concepts without relying on the team member who owns them.

Module 1 - Understand the Problem

Learn: node, link, packet, route, service, client, server; topology; directed vs undirected graph; weighted graph; path; connectivity; redundancy. Why it matters: the digital twin is fundamentally a graph $G=(V,E)$.

Example / mental model: Example: A-N1-N2-N3 can represent a simple service path. Add N4-N5 as a second route and you have redundancy.

Module 2 - Routing

Study BFS conceptually and Dijkstra in detail. Understand weighted shortest paths and dynamic edge costs.

Example / mental model: Example: A-B-D costs 15; A-C-E-D costs 11. The lower-cost route is selected. Then add congestion and risk to make the weights dynamic.

Module 3 - Distributed Systems

Understand failure, partial failure, fault tolerance, redundancy, failover, health checks, heartbeats, state consistency, cascading failure.

Example / mental model: Example: A fails -> traffic moves to B -> B overloads -> traffic moves to C -> C fails. Aegis should reason about second-order effects, not just the first failure.

Module 4 - Self-Healing Levels

Understand the difference between manual recovery, rule-based auto-remediation, and Aegis-style adaptive recovery.

Example / mental model: Traditional: Failure -> predefined rule -> predefined fix. Aegis: Observe -> Diagnose -> Generate plans -> Simulate -> Verify -> Deploy.

Module 5 - AI Agents

Learn observation, state, action, objective, constraint, planning.

Example / mental model: Aegis AI receives topology + telemetry, produces a structured recovery plan, and passes that plan to a safety gate. It is not the network administrator.

Module 6 - Structured AI Outputs

Understand why free-form natural language is insufficient for control. Learn JSON schemas, validation, malformed output handling, and constrained action spaces.

Example / mental model: Bad: "Maybe reroute through node 5." Good: a JSON plan with exact action types and node IDs.

Module 7 - Digital Twins

Understand a digital twin as a computational representation of the live system for safe experimentation.

Example / mental model: Production: N1 -> N2 -> N5. Twin: same state. Apply "disconnect N2" only in the twin and observe whether a valid alternate path exists.

Module 8 - Simulation

Understand state, event, transition, outcome, and counterfactual simulation.

Example / mental model: At $t=10$, N4 overloads. State transitions to degraded. The simulator calculates the resulting latency/availability.

Module 9 - Multi-Objective Optimization

Understand that there may be several competing objectives: availability, latency, congestion, security risk, migration cost.

Example / mental model: Plan A: 20 ms, 98% availability. Plan B: 30 ms, 99.9% availability. Aegis may prefer B depending on the objective weights and safety constraints.

Module 10 - Deterministic Safety

Understand invariants, constraints, policy checks, validation, safe action spaces, and fail-closed behavior.

Example / mental model: If a simulated plan drops availability below 99%, reject it. AI confidence never overrides a deterministic safety rule.

Module 11 - Security

Distinguish node failure from compromise. Understand anomaly, quarantine, secure rerouting, and suspicious behavior.

Example / mental model: Failure: N7 is dead. Compromise: N7 is alive but has abnormal traffic, packet loss, connection count, or resource use.

Module 12 - Fault Injection

Understand why resilience must be tested by intentionally breaking the system.

Example / mental model: Build: Kill Node, Break Link, Overload Node, Traffic Spike, Network Partition, Packet-Loss Spike.

5.1 Study sequence for today

- Phase 1 (60-90 min, all four): Networks, graphs, routing, failures, self-healing, distributed systems. Everyone draws the same network and explains what happens when a node or link fails.
- Phase 2 (60 min, all four): AI agent concepts, state/action/objective, structured output, digital twin, simulation, deterministic safety.
- Phase 3 (60 min, all four): Technology roles - NetworkX, FastAPI, React, WebSockets, Docker, Git. Focus on where each tool sits in the architecture.
- Phase 4 (role-specific): Split into ownership areas and study enough to begin implementation tonight.

5.2 Role-specific study

Member	Study topics today
Hrishi	System architecture; API design; state management; distributed-system failure modes; integration.
Sahil	NetworkX; graph modeling; Dijkstra; simulation; fault injection; telemetry generation.
Yyash	LLM APIs; structured output; prompt design; anomaly detection; AI planning.
Vikash	React; TypeScript basics; graph visualization; WebSockets; Pydantic / validation.

6. 5 September - Foundation Day & Night Build

By the end of today, everyone should know what is being built, what they personally own, how the modules communicate, and what the smallest functioning end-to-end system looks like.

6.1 Tonight: architecture freeze

Before writing serious code, sit together and physically draw the entire system. Answer these questions in order:

1. What is the current network state?
2. What can fail?
3. How do we detect a failure?
4. What information does the AI receive?
5. What actions can the AI propose?
6. How is an action represented?
7. How does the digital twin simulate it?
8. What metrics determine success?
9. What rules can NEVER be violated?
10. How does an approved plan change the network?

6.2 Foundation v0.1 - minimum working system

CREATE NETWORK

↓

SHOW NETWORK

↓

INJECT FAILURE

↓

UPDATE STATE

↓

SHOW FAILURE

Example:

Click “Kill N5”

↓

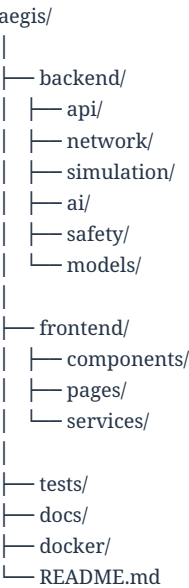
Backend changes N5 -> FAILED

↓

Frontend immediately shows N5 as failed

No AI is required at this stage. The point is to prove Frontend <-> Backend <-> Network Model integration before the harder components arrive.

6.3 Repository foundation



7. 6 September - Full Build Day

6 September is the heavy implementation day. The goal is a substantial prototype for the midnight review. The architecture should already be frozen from 5 September night.

Time / phase	Primary objective	Owners
Morning	Build network simulator: 15-30 nodes, topology, telemetry, fault injection.	Sahil + Hrishi integration
Late morning / noon	Make failure states observable in the UI. Ensure backend state changes propagate reliably.	Vikash + Hrishi
Afternoon	Build recovery pipeline with a mock/rule-based planner first: failure -> diagnosis -> candidates.	Yyash + Hrishi
Late afternoon	Implement digital twin: clone state, apply Plan A/B/C, compute metrics.	Sahil
Evening	Implement deterministic safety gate: reject unsafe plans, approve safe plans.	Vikash
Night	Replace mock planner with structured LLM diagnosis + recovery generation.	Yyash
Midnight review	Demonstrate failure -> diagnosis -> candidate plans -> simulation -> safety -> recovery.	All four

IMPORTANT SEQUENCING RULE

Build the entire pipeline with a deterministic/mock planner before connecting the LLM. This guarantees the project remains functional even if the LLM integration fails.

7.1 Target pipeline for the midnight review

```
FAILURE
↓
DIAGNOSIS
↓
RECOVERY PLANS
↓
DIGITAL TWIN
↓
SAFETY GATE
↓
RECOVERY
```

8. 7 September - Finish, Freeze, Test

Deadline	Objective	What must happen
Morning	Reliability	Fix crashes, invalid AI outputs, simulation bugs, race conditions, bad state transitions.
Afternoon	Presentation-quality UX	Improve UI, topology visualization, explanations, metrics, and recovery timeline.
By 8:00 PM	Feature freeze	No new major features after this point.
8:00-11:59 PM	Testing + demo rehearsal	Run the full scenario matrix, record expected/actual outcomes, reset the system between runs, rehearse the spoken demo.
11:59 PM	Eliminatory round ready	Only the most reliable build is allowed in the final demo environment.

8.1 Test matrix

Scenario	Expected behavior
Single node failure	Find an alternate route or safe workload relocation.
Link failure	Recompute viable route.
Node overload	Drain/migrate workload if a safe alternative exists.
Traffic surge	Redistribute traffic while respecting safety constraints.
Compromised node	Quarantine node and avoid it in new routes.
Multiple failures	Search for a surviving safe strategy.
Unsafe plan	Reject before deployment.
No safe plan	Do not deploy; surface the failure state clearly.
Malformed AI output	Schema validation rejects the plan.
Digital twin disagreement	Do not deploy if simulation validation fails.
Recovery causes cascade	Reject the plan or choose an alternative.

9. Final Demo Script - Make the Technical Innovation Visible

The eliminatory demonstration should be scripted. The judge should never have to imagine what the system is doing. The UI should reveal every important technical step.

DEMO 1 - Normal failure / autonomous recovery

1. Start with healthy network.
2. Kill N7.
3. Show failure detection.
4. AI diagnoses the failure.
5. AI generates Plan A, Plan B, Plan C.
6. Digital twin simulates all three.
7. Plan A -> rejected by safety or poor outcome.
8. Plan B -> safe and high-performing.
9. Safety gate -> APPROVED.
10. Apply Plan B.
11. Network recovers.
12. Show before/after metrics.

DEMO 2 - AI proposes an unsafe action

Network healthy
↓
Inject failure
↓
AI proposes: "Route everything through N3"
↓
Digital twin predicts N3 overload / availability drop
↓
Safety gate: REJECTED
↓
AI / planner produces another plan
↓
Plan passes simulation + deterministic policies
↓
APPROVED -> DEPLOY

WHY DEMO 2 MATTERS

This single scenario communicates the core innovation: Aegis does not need an AI that is always right. It needs a system that can remain safe when the AI is wrong.

DEMO 3 - Complex / cascading failure

Inject:
N4 overload + N7 failure + traffic spike
↓
Naive/local reaction could overload N8
↓
Aegis evaluates second-order effects in the digital twin
↓
Chooses the globally safer recovery strategy

The visual target: the judge sees the network break, the AI think, the twin test alternatives, the safety layer reject/approve, and the topology recover.

10. What We Say the Innovation Is

DO NOT CLAIM

“AI self-healing networks are new.” Self-healing and auto-remediation already exist. The credible claim is that Aegis explores a safety architecture where AI can generate recovery actions for previously unseen complex network states, while digital-twin simulation and deterministic policies prevent unsafe autonomous changes.

10.1 The clean innovation statement

Traditional automation is strong at known failures: detect a known condition, execute a known script. Unrestricted AI is flexible but dangerous because it may produce unexpected network changes. Aegis bridges the two: AI supplies adaptive planning, while simulation and deterministic policies constrain production authority.

Traditional Level-2 automation:

Failure -> predefined rule -> predefined recovery

Aegis prototype:

Failure -> AI diagnosis -> candidate plans -> digital-twin simulation

-> deterministic safety gate -> approved recovery

Long-term vision:

Observe -> Diagnose -> Predict -> Generate Plans -> Simulate

-> Verify -> Deploy -> Learn -> Decentralize

10.2 “Separate intelligence from authority”

This should be a central phrase in the technical narrative. The AI can be probabilistic and occasionally wrong. The infrastructure must remain deterministic about what is permissible.

AI = flexible

Policy = strict

Simulator = evidence

Together -> safe autonomy

11. Future Scope & Scalability

The hackathon prototype should be framed as the first layer of a much larger system, not as a claim to have built production-grade autonomous networking.

Stage	Potential evolution
MVP	Simulated cluster; AI recovery planner; digital twin; deterministic safety engine; real-time dashboard.
Advanced planning	Reinforcement learning or search-based recovery policies; better multi-objective optimization.
Security	Compromised-node detection; threat-aware routing; autonomous quarantine and secure path planning.
Distributed control	Peer-to-peer or hierarchical recovery where regional controllers can operate despite central failure.
Infrastructure integration	SDN, Kubernetes, cloud orchestration, container migration, real network telemetry.
Edge / 5G / 6G	Geographically distributed edge nodes and latency-sensitive recovery.
Autonomous infrastructure	A system that observes, diagnoses, predicts, plans, safely validates, and repairs infrastructure with minimal human intervention.

11.1 Scalability model

10 nodes
↓
100 nodes
↓
1,000 nodes
↓
10,000 nodes

Potential architecture:

```
      Global Controller
      /   |   \
    Region A Region B Region C
      ↓     ↓     ↓
    local  local  local
    workers workers workers
```

Candidate evaluation is naturally parallel: many recovery plans can be simulated independently. This provides a clear path toward parallel workers, distributed telemetry processing, hierarchical control planes, and cloud-scale experimentation.

12. 8 September - If We Qualify: Presentation Plan

Slide	Purpose	Core content
1. Aegis	Hook	Safety-Gated Autonomous Network Recovery.
2. Problem	Establish gap	Known failure -> predefined script. What happens when the failure is unseen or cascading?
3. Insight	Explain architecture choice	AI adds adaptability; unrestricted AI is unsafe. Separate intelligence from authority.
4. Architecture	Show technical depth	Telemetry -> diagnosis -> planning -> digital twin -> safety gate -> deployment.
5. Workflow	Show one failure end-to-end	Observe -> Diagnose -> Plan -> Simulate -> Verify -> Deploy.
6. Implementation	Prove engineering	Python, NetworkX, FastAPI, React, WebSockets, LLM, simulation, policy engine, Docker.
7. Innovation	State contribution	Recovery actions are generated adaptively but never directly deployed; they must survive simulation and deterministic policy checks.
8. Results	Prove performance	Use real experiment numbers: recovery time, availability, latency, packet loss, unsafe-plan rejection, cascade avoidance.
9. Future	Show roadmap	RL, P2P recovery, threat-aware routing, SDN/Kubernetes/cloud/edge.
10. Demo / close	End strongly	Show the system break, think, reject/approve, and recover.

13. Technical Concepts - Quick Reference

Concept	Meaning
Graph	A set of vertices/nodes connected by edges/links.
Weighted graph	A graph where each edge has a cost such as latency or risk.
Dijkstra	Shortest-path algorithm for non-negative edge costs.
Health check	A mechanism for determining whether a node/service is responsive and healthy.
Heartbeat	Periodic liveness signal between nodes/services.
Fault tolerance	Ability to continue operating when components fail.
Cascading failure	A local failure causes secondary overloads/failures elsewhere.
Anomaly detection	Identifying network behavior that deviates from expected patterns.
Digital twin	A computational representation of a real system used to study state and interventions safely.
Counterfactual	“What would happen if we applied this change?” tested without changing the live state.
Policy / invariant	A deterministic rule describing a condition that must always hold.
Fail-closed	When validation or confidence fails, the system refuses to deploy rather than taking an unsafe action.
Structured output	Machine-readable AI output following a fixed schema rather than free-form prose.
Multi-objective optimization	Choosing among solutions while trading off several objectives such as availability, latency, and risk.
Telemetry	Operational measurements describing system behavior over time.
Fault injection	Deliberately creating failures to test resilience.

13.1 Example metric model

Candidate Plan Score =

- w1 * Availability
- w2 * LatencyIncrease
- w3 * Congestion
- w4 * SecurityRisk

But safety comes first:

IF Availability < 0.99 -> REJECT

IF MaxLatencyIncrease > 0.30 -> REJECT

IF UsesQuarantinedNode -> REJECT

IF NoValidPath -> REJECT

14. Team Checklists

14.1 End of 5 September

- All four can draw and explain the full Aegis pipeline.
- Tech stack is fixed.
- Repository created with module folders.
- Network State schema is fixed and documented.
- API contracts are agreed.
- Each person knows their subsystem and handoff points.
- Foundation v0.1 can create a network, inject a failure, update state, and show the failure.
- No major architecture questions remain unresolved overnight.

14.2 End of 6 September / Midnight Review

- 15-30 node simulated network exists.
- Fault injection works.
- Telemetry is visible.
- AI diagnosis works with structured output.
- At least 2-3 recovery plans can be generated.
- Digital twin simulates candidate plans.
- Safety gate can reject and approve plans.
- At least one end-to-end recovery scenario works reliably.

14.3 By 8 PM on 7 September

- Feature set frozen.
- All three demo scenarios work.
- Malformed AI output is handled safely.
- Unsafe plans are rejected.
- No-safe-plan condition is handled.
- UI explains the pipeline clearly.
- Metrics are captured for before/after comparison.
- Build can be restarted from a clean environment.

14.4 Final mental checklist before judging

What failed?

What evidence did Aegis observe?

What did the AI infer?

What plans did it generate?

How did the digital twin evaluate them?

Which safety rules were checked?

Why was the chosen plan safe?

What changed in the network?

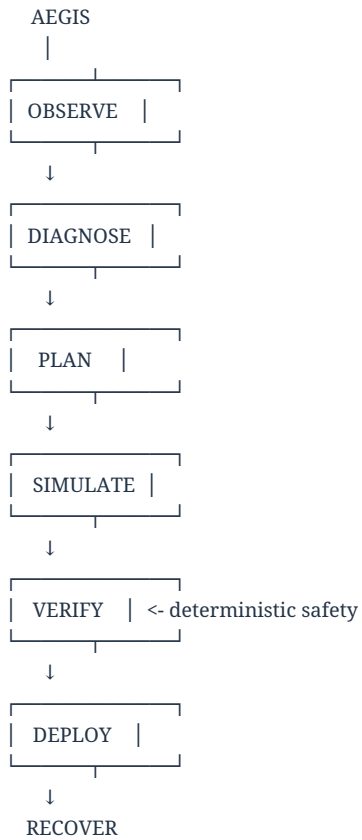
Did the system recover?

What did we measure?

15. The One-Page Aegis Memory Sheet

REMEMBER THIS SENTENCE

Aegis is a simulated autonomous network-recovery engine where AI proposes recovery strategies, a digital twin tests them, and deterministic safety policies decide what can reach production.



AI = flexible

Policy = strict

Simulator = evidence

NO AI OUTPUT TOUCHES THE NETWORK DIRECTLY.

Build the smallest reliable system first. Then make it impressive.